

Topic 1: Introduction

Design and Analysis of Algorithms Laboratory

Exercise 1.1: First Palindromic String in an Array

Aim:

To find and return the first palindromic string in a given array of strings. If no such string exists, return an empty string.

Python Code:

```
def first_palindrome(words):
    """
    Returns the first string in words that reads the same forward and backward.
    Time Complexity: O(n * k), Space Complexity: O(1)
    """
    for word in words:
        if word == word[::-1]:
            return word
    return ""

# Test Cases
words1 = ["abc", "car", "ada", "racecar", "cool"]
print(f'Input: {words1} -> Output: "{first_palindrome(words1)}"')

words2 = ["notapalindrome", "racecar"]
print(f'Input: {words2} -> Output: "{first_palindrome(words2)}"')
```

Sample Input:

```
words = ["abc", "car", "ada", "racecar", "cool"]
words = ["notapalindrome", "racecar"]
```

Sample Output:

```
Output: "ada"
Output: "racecar"
```

Exercise 1.2: Intersection Values Between Two Arrays

Aim:

To calculate the number of indices in nums1 whose elements exist in nums2, and the number of indices in nums2 whose elements exist in nums1.

Python Code:

```
def find_intersection_values(nums1, nums2):
    """
    Calculates:
    answer1: count of elements in nums1 that appear in nums2
    answer2: count of elements in nums2 that appear in nums1
    Time Complexity: O(n + m), Space Complexity: O(n + m)
    """
    set1 = set(nums1)
    set2 = set(nums2)
    ans1 = sum(1 for x in nums1 if x in set2)
    ans2 = sum(1 for x in nums2 if x in set1)
    return [ans1, ans2]

# Test Cases
print(find_intersection_values([2, 3, 2], [1, 2]))
print(find_intersection_values([4, 3, 2, 3, 1], [2, 2, 5, 2, 3, 6]))
```

Sample Input:

```
nums1 = [2, 3, 2], nums2 = [1, 2]
nums1 = [4, 3, 2, 3, 1], nums2 = [2, 2, 5, 2, 3, 6]
```

Sample Output:

```
Output: [2, 1]
Output: [3, 4]
```

Exercise 1.3: Sum of Squares of Distinct Counts in Subarrays

Aim:

To compute the sum of squares of distinct element counts across all possible contiguous subarrays of an integer array.

Python Code:

```
def sum_counts(nums):
    """
    Computes the sum of (distinct_count(sub))^2 for all subarrays.
    Time Complexity: O(n^2), Space Complexity: O(n)
    """
    n = len(nums)
    total = 0
    for i in range(n):
        distinct = set()
        for j in range(i, n):
            distinct.add(nums[j])
            total += len(distinct) ** 2
    return total

# Test Cases
print(f"nums = [1, 2, 1] -> Output: {sum_counts([1, 2, 1])}")
print(f"nums = [1, 1] -> Output: {sum_counts([1, 1])}")
```

Sample Input:

```
nums = [1, 2, 1]
nums = [1, 1]
```

Sample Output:

```
Output: 15
Output: 3
```

Exercise 1.4: Count Equal and Divisible Pairs in an Array**Aim:**

To find the number of pairs (i, j) with $0 \leq i < j < n$ such that $\text{nums}[i] == \text{nums}[j]$ and $(i * j)$ is divisible by k .

Python Code:

```
def count_pairs(nums, k):
    """
    Finds pairs (i, j) where nums[i] == nums[j] and (i * j) % k == 0.
    Time Complexity: O(n^2), Space Complexity: O(1)
    """
    n = len(nums)
    count = 0
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1
    return count

# Test Cases
print(f"nums = [3,1,2,2,2,1,3], k = 2 -> Pairs: {count_pairs([3,1,2,2,2,1,3], 2)}")
print(f"nums = [1,2,3,4], k = 1 -> Pairs: {count_pairs([1,2,3,4], 1)}")
```

Sample Input:

```
nums = [3, 1, 2, 2, 2, 1, 3], k = 2
nums = [1, 2, 3, 4], k = 1
```

Sample Output:

```
Output: 4
Output: 0
```

Exercise 1.5: Find Maximum Element with Least Time Complexity

Aim:

To determine the maximum element in a list of numbers using the optimal linear time complexity $O(n)$.

Python Code:

```
def find_max(arr):  
    """  
    Finds the maximum element in a list in  $O(n)$  time and  $O(1)$  space.  
    """  
    if not arr:  
        return None  
    max_val = arr[0]  
    for x in arr[1:]:  
        if x > max_val:  
            max_val = x  
    return max_val  
  
# Test Cases  
test_cases = [  
    [1, 2, 3, 4, 5],  
    [7, 7, 7, 7, 7],  
    [-10, 2, 3, 4, 5]  
]  
for tc in test_cases:  
    print(f"Input: {tc} -> Expected Output: {find_max(tc)}")
```

Sample Input:

Input: {1, 2, 3, 4, 5}
Input: {7, 7, 7, 7, 7}
Input: {-10, 2, 3, 4, 5}

Sample Output:

Expected Output: 5
Expected Output: 7
Expected Output: 5

Exercise 1.6: Efficient Sort and Find Maximum Element

Aim:

To sort a list of numbers using an efficient sorting algorithm and subsequently retrieve the maximum element.

Python Code:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    res = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            res.append(left[i]); i += 1
        else:
            res.append(right[j]); j += 1
    res.extend(left[i:])
    res.extend(right[j:])
    return res

def process_and_find_max(arr):
    if not arr:
        return "None (List is empty)"
    sorted_arr = merge_sort(arr)
    return sorted_arr[-1]

# Test Cases
print("Test 1 (Empty List):", process_and_find_max([]))
print("Test 2 (Single Element):", process_and_find_max([5]))
print("Test 3 (All Same):", process_and_find_max([3, 3, 3, 3, 3]))
```

Sample Input:

1. Input: []
2. Input: [5]
3. Input: [3, 3, 3, 3, 3]

Sample Output:

1. Expected Output: None or list is empty
2. Expected Output: 5
3. Expected Output: 3

Exercise 1.7: Extract Unique Elements and Space Complexity Analysis

Aim:

To construct a new list containing only the unique elements from an input list, preserving order, and analyze space complexity.

Python Code:

```
def get_unique_elements(arr):
    """
    Extracts unique elements while preserving original order.
    Space Complexity: O(n) auxiliary space to store seen set and result list.
    Time Complexity: O(n) average lookup with hash set.
    """
    seen = set()
    unique_list = []
    for x in arr:
        if x not in seen:
            seen.add(x)
            unique_list.append(x)
    return unique_list

# Test Cases
print(get_unique_elements([3, 7, 3, 5, 2, 5, 9, 2]))
print(get_unique_elements([-1, 2, -1, 3, 2, -2]))
print(get_unique_elements([1000000, 999999, 1000000]))
```

Sample Input:

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Input: [-1, 2, -1, 3, 2, -2]

Input: [1000000, 999999, 1000000]

Sample Output:

Expected Output: [3, 7, 5, 2, 9]

Expected Output: [-1, 2, 3, -2]

Expected Output: [1000000, 999999]

Space Complexity: O(n)

Exercise 1.8: Bubble Sort Algorithm and Big-O Analysis

Aim:

To sort an array of integers using the Bubble Sort technique and analyze its time and space complexity.

Python Code:

```
def bubble_sort(arr):
    """
    Bubble Sort implementation.
    Time Complexity:
    - Worst case:  $O(n^2)$  when array is reverse sorted
    - Best case:  $O(n)$  when array is already sorted (with swap flag)
    - Average case:  $O(n^2)$ 
    Space Complexity:  $O(1)$  in-place
    """
    a = arr.copy()
    n = len(a)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    return a

arr = [64, 34, 25, 12, 22, 11, 90]
print(f"Original: {arr}")
print(f"Sorted: {bubble_sort(arr)}")
```

Sample Input:

Input: [64, 34, 25, 12, 22, 11, 90]

Sample Output:

Output: [11, 12, 22, 25, 34, 64, 90]

Time Complexity: $O(n^2)$ worst/avg, $O(n)$ best. Space: $O(1)$

Exercise 1.9: Binary Search in a Sorted Array with Big-O Analysis

Aim:

To determine whether a given key exists in a sorted array using Binary Search and display its index, along with Big-O complexity analysis.

Python Code:

```
def binary_search(arr, key):
    """
    Searches for key in sorted array arr.
    Returns 0-based index or -1 if not found.
    Time Complexity: O(log n), Space Complexity: O(1)
    """
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Test Cases
arr = [-9, 3, 4, 6, 8, 9, 10, 30] # Sorted array
for k in [10, 100]:
    pos = binary_search(arr, k)
    if pos != -1:
        print(f"Element {k} is found at position {pos}")
    else:
        print(f"Element {k} is not found")
```

Sample Input:

```
arr = [-9, 3, 4, 6, 8, 9, 10, 30], KEY = 10
arr = [-9, 3, 4, 6, 8, 9, 10, 30], KEY = 100
```

Sample Output:

```
Output: Element 10 is found at position 6 (0-indexed)
Output: Element 100 is not found
Time Complexity: O(log n), Space: O(1)
```

Exercise 1.10: $O(n \log n)$ Sort with $O(1)$ Space (Heap Sort)**Aim:**

To sort an integer array in ascending order without using built-in functions in $O(n \log n)$ time and $O(1)$ auxiliary space using Heap Sort.

Python Code:

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(nums):
    """
    Sorts nums in-place using Heap Sort.
    Time Complexity:  $O(n \log n)$ , Space Complexity:  $O(1)$  auxiliary
    """
    n = len(nums)
    # Build max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(nums, n, i)
    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        nums[0], nums[i] = nums[i], nums[0]
        heapify(nums, i, 0)
    return nums

# Test Cases
print(heap_sort([5, 2, 3, 1]))
print(heap_sort([5, 1, 1, 2, 0, 0]))
```

Sample Input:

Input: [5, 2, 3, 1]

Input: [5, 1, 1, 2, 0, 0]

Sample Output:

Output: [1, 2, 3, 5]

Output: [0, 0, 1, 1, 2, 5]

Time: $O(n \log n)$, Auxiliary Space: $O(1)$

Exercise 1.11: Out of Boundary Paths on Grid

Aim:

To find the number of ways to move a ball out of an $m \times n$ grid boundary in exactly N steps from starting cell (i, j) .

Python Code:

```
def find_paths(m, n, N, start_row, start_col):
    """
    Dynamic programming solution for out of boundary paths.
    Time Complexity:  $O(N * m * n)$ , Space Complexity:  $O(m * n)$ 
    """
    MOD = 10**9 + 7
    dp = [[0] * n for _ in range(m)]
    dp[start_row][start_col] = 1
    out_paths = 0

    for step in range(N):
        next_dp = [[0] * n for _ in range(m)]
        for r in range(m):
            for c in range(n):
                if dp[r][c] > 0:
                    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                        nr, nc = r + dr, c + dc
                        if 0 <= nr < m and 0 <= nc < n:
                            next_dp[nr][nc] = (next_dp[nr][nc] + dp[r][c]) % MOD
                        else:
                            out_paths = (out_paths + dp[r][c]) % MOD
        dp = next_dp

    return out_paths

# Test Cases
print(f'm=2, n=2, N=2, i=0, j=0 -> Output: {find_paths(2, 2, 2, 0, 0)}')
print(f'm=1, n=3, N=3, i=0, j=1 -> Output: {find_paths(1, 3, 3, 0, 1)}')
```

Sample Input:

Input: $m = 2, n = 2, N = 2, i = 0, j = 0$

Input: $m = 1, n = 3, N = 3, i = 0, j = 1$

Sample Output:

Output: 6

Output: 12

Exercise 1.12: House Robber II (Circular Houses)

Aim:

To determine the maximum money a robber can steal from houses arranged in a circle such that no two adjacent houses are robbed.

Python Code:

```
def rob_linear(houses):
    prev1, prev2 = 0, 0
    for num in houses:
        prev1, prev2 = max(prev2 + num, prev1), prev1
    return prev1

def rob_circular(nums):
    """
    Time Complexity: O(n), Space Complexity: O(1)
    """
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]
    return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))

# Test Cases
print(f'nums = [2, 3, 2] -> Max Money: {rob_circular([2, 3, 2])}')
print(f'nums = [1, 2, 3, 1] -> Max Money: {rob_circular([1, 2, 3, 1])}')
```

Sample Input:

Input: nums = [2, 3, 2]

Input: nums = [1, 2, 3, 1]

Sample Output:

Output: The maximum money you can rob without alerting the police is 3

Output: The maximum money you can rob without alerting the police is 4

Exercise 1.13: Climbing Stairs Problem

Aim:

To compute the total number of distinct ways to climb to the top of a staircase of n steps taking 1 or 2 steps at a time.

Python Code:

```
def climb_stairs(n):
    """
    Dynamic programming with O(1) space (Fibonacci sequence).
    Time Complexity: O(n), Space Complexity: O(1)
    """
    if n <= 2:
        return n
    a, b = 1, 2
    for _ in range(3, n + 1):
        a, b = b, a + b
    return b

# Test Cases
print(f'n = 4 -> Ways: {climb_stairs(4)}')
print(f'n = 3 -> Ways: {climb_stairs(3)}')
```

Sample Input:

Input: n = 4

Input: n = 3

Sample Output:

Output: 5

Output: 3

Exercise 1.14: Unique Paths in an m x n Grid**Aim:**

To compute the total number of unique paths from the top-left corner to the bottom-right corner of an m x n grid moving only right or down.

Python Code:

```
import math

def unique_paths(m, n):
    """
    Using Combinatorics: (m+n-2) choose (m-1)
    Time Complexity: O(min(m, n)), Space Complexity: O(1)
    """
    return math.comb(m + n - 2, m - 1)

# Test Cases
print(f'm = 7, n = 3 -> Unique Paths: {unique_paths(7, 3)}')
print(f'm = 3, n = 2 -> Unique Paths: {unique_paths(3, 2)}')
```

Sample Input:

Input: m = 7, n = 3

Input: m = 3, n = 2

Sample Output:

Output: 28

Output: 3

Exercise 1.15a: Positions of Large Groups in a String

Aim:

To find the starting and ending index intervals of all consecutive character groups of length 3 or more in a string.

Python Code:

```
def large_group_positions(s):
    """
    Time Complexity: O(n), Space Complexity: O(1) auxiliary
    """
    res = []
    i = 0
    n = len(s)
    while i < n:
        j = i
        while j < n and s[j] == s[i]:
            j += 1
        if j - i >= 3:
            res.append([i, j - 1])
        i = j
    return res

# Test Cases
print('s = "abbxxxxzzy" ->', large_group_positions("abbxxxxzzy"))
print('s = "abc" ->', large_group_positions("abc"))
```

Sample Input:

Input: s = "abbxxxxzzy"

Input: s = "abc"

Sample Output:

Output: [[3, 6]]

Output: []

Exercise 1.15b: Conway's Game of Life Cellular Automaton

Aim:

To simulate Conway's Game of Life and generate the next state of an $m \times n$ grid according to underpopulation, survival, overpopulation, and reproduction rules.

Python Code:

```
def game_of_life(board):
    """
    Computes the next state in-place using bit manipulation or copy.
    Time Complexity:  $O(m * n)$ , Space Complexity:  $O(m * n)$ 
    """
    m, n = len(board), len(board[0])
    next_board = [[0] * n for _ in range(m)]

    for r in range(m):
        for c in range(n):
            live_neighbors = 0
            for dr in [-1, 0, 1]:
                for dc in [-1, 0, 1]:
                    if dr == 0 and dc == 0:
                        continue
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < m and 0 <= nc < n and board[nr][nc] == 1:
                        live_neighbors += 1

            if board[r][c] == 1:
                if live_neighbors in (2, 3):
                    next_board[r][c] = 1
            else:
                next_board[r][c] = 0
            if live_neighbors == 3:
                next_board[r][c] = 1
            else:
                next_board[r][c] = 0
    return next_board

# Test Cases
board1 = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
print("Board 1 Next State:", game_of_life(board1))

board2 = [[1,1],[1,0]]
print("Board 2 Next State:", game_of_life(board2))
```

Sample Input:

```
board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
board = [[1,1],[1,0]]
```

Sample Output:

```
Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]
Output: [[1,1],[1,1]]
```

Exercise 1.16: Champagne Tower Simulation

Aim:

To simulate pouring champagne into a glass pyramid and determine how full the glass at (query_row, query_glass) is.

Python Code:

```
def champagne_tower(poured, query_row, query_glass):
    """
    Simulates pouring into glass pyramid.
    Time Complexity: O(query_row^2), Space Complexity: O(query_row^2)
    """
    tower = [[0.0] * (r + 1) for r in range(query_row + 1)]
    tower[0][0] = float(poured)

    for r in range(query_row):
        for c in range(r + 1):
            if tower[r][c] > 1.0:
                excess = (tower[r][c] - 1.0) / 2.0
                tower[r + 1][c] += excess
                tower[r + 1][c + 1] += excess

    return f"{min(1.0, tower[query_row][query_glass]):.5f}"

# Test Cases
print("poured=1, r=1, c=1 ->", champagne_tower(1, 1, 1))
print("poured=2, r=1, c=1 ->", champagne_tower(2, 1, 1))
```

Sample Input:

Input: poured = 1, query_row = 1, query_glass = 1

Input: poured = 2, query_row = 1, query_glass = 1

Sample Output:

Output: 0.00000

Output: 0.50000